UNDERSTAND AI

How AI Answers

In Vector Space, you watched a token's final vector land in its neighborhood on the map. That landing was about meaning: the model understanding the words you put in. Now we move to the next step: how AI answers you.

Start with your phone. As you type a text, it suggests the next word: three choices above the keyboard, picked from the last word or two. And everyone gets pretty much the same suggestions.

See you _____.

soon | tomorrow | later

AI follows the same basic logic as your phone's suggestions, but, as you might expect, it goes **way deeper**. Let's take a simple question and watch AI build its answer.

What should I name my new dog?

TOKENS
Chops the text into tokens and looks up Token IDs.

What	3923
should	1288
I	358
name	836
my	856
new	502
dog	5679
?	30

Now: the text is pieces the model can look up.

POSITIONS
Marks each token's position.

What	#1
should	#2
I	#3
name	#4
my	#5
new	#6
dog	#7
?	#8

Now: word order can't get lost.

STARTING MEANING
Looks up the vector for each token.

What	[0.42,−1.15,0.63,…]
should	[−0.87,0.21,1.44,…]
I	[1.33,−0.06,−0.71,…]
name	[0.27,1.91,−0.38,…]
my	[−0.52,0.68,0.19,…]
new	[0.95,−1.24,0.57,…]
dog	[−1.08,0.74,1.62,…]
?	[0.11,−0.39,−0.84,…]

Now: each token carries its dictionary meaning, the same vector in every sentence.

THROUGH LAYERS
Runs Attention & Transformation.

What	[1.87,0.42,−2.31,…]
should	[−0.19,2.66,0.85,…]
I	[2.14,−1.52,0.98,…]
name	[−2.43,1.17,2.05,…]
my	[1.29,−0.77,−1.94,…]
new	[−1.61,2.38,0.44,…]
dog	[2.72,−2.15,1.83,…]
?	[−0.96,1.49,−2.58,…]

Now: the numbers changed: every vector now knows the **question so far**.

WHERE WE STAND

Look at what the model has now. The question isn't words anymore. It's eight rich vectors, each one carrying what its token means inside this exact question. **Meaning: established.**

Now, one detail changes everything: **the last token matters most**. Why? The model is about to write, and the next word goes in exactly one place: right after the last token. So that's the vector the model reads to pick it. And the layers have been preparing that vector all along: in every layer, attention folds the earlier tokens into the later ones, so **the last token's vector carries the entire question**.

By the final layer, the last token isn't really about the question mark (?) anymore. Its job has changed. Remember what the model practiced during training: predicting the next word, over and over, billions of times. **Training tuned the layers to turn what the text *means* into where the next word *lives*.** So the vector carrying the question gets pushed across the map, and it lands in the neighborhood of the words most likely to come next: the first word of the answer.

THE ANSWER, TOKEN BY TOKEN

Now watch it happen. Each row below is one token of the answer, and in this example every token is a whole word. The model looks at the last token (the first column), checks which neighborhood its vector landed in, and takes the best-scoring word there. That word gets typed, joins the context, and becomes the new last token. Then the next row repeats the move. That move is called **prediction**.

What should I name my new dog?

LAST TOKEN	REPLY SO FAR	NEIGHBORHOOD	TOP PREDICTIONS
?	___	*Reply starters*	You 18% · A 14% · Great 9%
You	You ___	*Ways to suggest*	could 24% · can 12% · should 10%
could	You could ___	*Naming verbs*	name 31% · call 19% · try 7%
name	You could name ___	*Who gets named*	him 45% · your 21% · the 8%
him	You could name him …	*Dog names*	Spot 22% · Max 17% · Buddy 14%

✓ **You could name him Spot.**

You asked for a name for your new dog, but the model didn't start with a name. It started with You, the most likely first word of a reply. Read the Neighborhood column from top to bottom and you can watch the sentence close in: reply starters, ways to suggest, naming verbs, who gets named, and only then dog names. The name arrived when the sentence had built a slot that only a name could fill. Five rows, one move, and a context that grew by one word each time.

THE RANKED LIST

One more idea to clarify. The table shows three predictions per row, but that's just the top of the list. In reality, the model assigns a probability to every token in its vocabulary, hundreds of thousands of them, and ranks them all. That's what "landing in a neighborhood" really is: the neighborhood is the top of the ranking. Here's the name slot under the microscope: the moment Spot arrived.

Score every token: the name slot

You could name him _____

Spot	22%
Max	17%
Buddy	14%
Rex	9%
Biscuit	6%
other tokens	32%

Why is "other tokens" the biggest? It's the rest of the vocabulary sharing that 32%; no single one of them comes close to Spot. The exact numbers are illustrative.

The model takes the top of the list and types it: Spot.

And that's the whole machine. Everything you just watched, from tokens to vectors to a ranked list to the next word, has a name: **inference**. It's what the model does every time it answers you.

Every answer is built one token at a time.

The whole run is called inference.